



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Práctica Guiada 9: Procedimientos Almacenados y Paginación Web Dinámica con Hibernate, Spring Boot y HTML5

Resumen

Esta práctica guiada consolida los temas avanzados de persistencia del **Tema 8.7.1 (Hibernate Avanzado)**, integrando la ejecución de **procedimientos almacenados** (*Stored Procedures*) y la **paginación de datos a nivel de base de datos** (*Pageable, Page<T>*) con una interfaz web en **HTML5, CSS3 y JavaScript asíncrono (Fetch API)**.

A lo largo del laboratorio, usted construirá un módulo de consulta de facturación empresarial. Aprenderá a mapear llamadas a procedimientos almacenados en la base de datos, estructurar consultas paginadas en Spring Boot para evitar la carga masiva en memoria RAM, exponer endpoints RESTful paginados y consumir dichos resultados desde una tabla HTML interactiva con

botones de navegación (Anterior, Siguiente, Primera, Última) y selector de tamaño de página.

Metadatos del Laboratorio

- **Tiempo estimado:** 4 horas.
- **Herramientas requeridas:** Java 21, Spring Boot 3.x, H2 Database (o SQL Server), Maven, VS Code / IntelliJ IDEA, Navegador Web (Chrome/Firefox).

Metas de Aprendizaje:

1. Configurar y ejecutar procedimientos almacenados nativos mediante la anotación `@Procedure` y `EntityManager` de JPA.
2. Implementar paginación relacional eficiente utilizando `Pageable`, `PageRequest` y `Page<T>` en Spring Data JPA.
3. Construir un controlador RESTful que retorne payloads JSON paginados estandarizados.
4. Desarrollar una interfaz de usuario cliente en HTML5 y Vanilla JavaScript que consuma datos paginados mediante `fetch()` y actualice dinámicamente el DOM.
5. Diagnosticar y resolver errores comunes como el desajuste de índices de página (base 0 vs base 1) y advertencias de paginación en memoria (HHH000104).

Conceptos Clave (El 'Qué')

1. Procedimientos Almacenados (`@Procedure`)

Un procedimiento almacenado es una rutina precompilada en el motor de base de datos. Spring Data JPA permite invocar estos procedimientos mediante la anotación `@Procedure` en interfaces `@Repository`, mapeando parámetros de entrada (IN) y salida (OUT).

2. Paginación Relacional (`Pageable` y `Page<T>`)

La interfaz Pageable transmite el número de página deseado, el tamaño de lote y los criterios de ordenamiento. Hibernate traduce esto en cláusulas SQL nativas (`LIMIT / OFFSET` o `OFFSET ... FETCH NEXT`), evitando saturar la memoria de la JVM.

3. Consumo Asíncrono Web (Fetch API)

El navegador realiza peticiones HTTP asíncronas (`GET /api/v1/facturas?page=0&size=10`) sin recargar la página completa, renderizando únicamente las filas de la tabla y recalculando los botones de la barra de paginación.

Parte 1: Práctica Guiada Paso a Paso

Paso 1: Configuración de la Entidad `Factura` y Script SQL con Procedimiento Almacenado

Cree la entidad `Factura.java` en el paquete `domain`:

```
package cr.ac.ucr.ie.domain;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.LocalDate;

@Entity
@Table(name = "Factura")
public class Factura {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 20, unique = true)
    private String numeroFactura;

    @Column(nullable = false, length = 100)
    private String cliente;

    @Column(nullable = false, precision = 12, scale = 2)
    private BigDecimal montoTotal;
```

```

    @Column(nullable = false)
    private LocalDate fecha;

    @Column(nullable = false, length = 20)
    private String estado;

    public Factura() {}

    public Factura(String numeroFactura,
                   String cliente,
                   BigDecimal montoTotal,
                   LocalDate fecha,
                   String estado) {
        this.numeroFactura = numeroFactura;
        this.cliente = cliente;
        this.montoTotal = montoTotal;
        this.fecha = fecha;
        this.estado = estado;
    }

    public Long getId() {
        return id;
    }

    public String getNumeroFactura() {
        return numeroFactura;
    }

    public String getCliente() {
        return cliente;
    }

    public BigDecimal getMontoTotal() {
        return montoTotal;
    }

    public LocalDate getFecha() {
        return fecha;
    }

    public String getEstado() {
        return estado;
    }
}

```

Cree el archivo de inicialización SQL src/main/resources/schema.sql:

```
-- Script de esquema y Stored Procedure
CREATE TABLE IF NOT EXISTS Factura (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    numero_factura VARCHAR(20) NOT NULL UNIQUE,
    cliente VARCHAR(100) NOT NULL,
    monto_total DECIMAL(12,2) NOT NULL,
    fecha DATE NOT NULL,
    estado VARCHAR(20) NOT NULL
);

-- Stored Procedure compatible con H2 / SQL
CREATE ALIAS IF NOT EXISTS SP_OBTENER_FACTURAS AS $$
import java.sql.*;
@CODE
ResultSet spObtenerFacturas(Connection conn,
                             String pEstado)
throws SQLException {
    String sql = "SELECT * FROM Factura " +
                "WHERE estado = ? " +
                "ORDER BY fecha DESC";
    PreparedStatement ps =
        conn.prepareStatement(sql);
    ps.setString(1, pEstado);
    return ps.executeQuery();
}
$$;
```

Cree el archivo de semillas src/main/resources/data.sql:

```
INSERT INTO Factura
(numero_factura, cliente, monto_total, fecha, estado)
VALUES ('FAC-001', 'Alfa S.A.', 1500.00, '2026-09-01', 'PAGADA');

INSERT INTO Factura
(numero_factura, cliente, monto_total, fecha, estado)
VALUES ('FAC-002', 'Beta Ltda.', 450.50, '2026-09-02', 'PENDIENTE');

INSERT INTO Factura
(numero_factura, cliente, monto_total, fecha, estado)
VALUES ('FAC-003', 'Gamma Inc.', 3200.00, '2026-09-03', 'PAGADA');

INSERT INTO Factura
```

```

(numero_factura, cliente, monto_total, fecha, estado)
VALUES ('FAC-004', 'Delta Corp.', 890.00, '2026-09-04', 'PENDIENTE');

INSERT INTO Factura
(numero_factura, cliente, monto_total, fecha, estado)
VALUES ('FAC-005', 'Epsilon S.A.', 2100.75, '2026-09-05', 'PAGADA');

INSERT INTO Factura
(numero_factura, cliente, monto_total, fecha, estado)
VALUES ('FAC-006', 'Zeta Ltda.', 670.20, '2026-09-06', 'ANULADA');

INSERT INTO Factura
(numero_factura, cliente, monto_total, fecha, estado)
VALUES ('FAC-007', 'Eta Corp.', 4500.00, '2026-09-07', 'PAGADA');

INSERT INTO Factura
(numero_factura, cliente, monto_total, fecha, estado)
VALUES ('FAC-008', 'Theta S.A.', 120.00, '2026-09-08', 'PENDIENTE');

```

Paso 2: Creación del DTO y Repositorio de Spring Data JPA

Cree el DTO `FacturaDTO.java` en `dto`:

```

package cr.ac.ucr.ie.dto;

import java.math.BigDecimal;
import java.time.LocalDate;

public record FacturaDTO(
    Long id,
    String numeroFactura,
    String cliente,
    BigDecimal montoTotal,
    LocalDate fecha,
    String estado
) {}

```

Cree el repositorio `FacturaRepository.java` en `data`:

```

package cr.ac.ucr.ie.data;

```

```

import cr.ac.ucr.ie.domain.Factura;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.query.Procedure;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface FacturaRepository
    extends JpaRepository<Factura, Long> {

    // Invocacion de Stored Procedure nativo
    @Procedure(procedureName = "SP_OBTENER_FACTURAS")
    List<Factura> obtenerPorEstado(@Param("pEstado") String est);

    // Consulta paginada por nombre de cliente
    Page<Factura> findByClienteContainingIgnoreCase(
        String cliente,
        Pageable pageable
    );

    Page<Factura> findByEstado(
        String estado,
        Pageable pageable
    );
}

```

Paso 3: Capa de Servicio de Negocio (**FacturaService.java**)

Cree la clase FacturaService.java en business:

```

package cr.ac.ucr.ie.business;

import cr.ac.ucr.ie.data.FacturaRepository;
import cr.ac.ucr.ie.domain.Factura;
import cr.ac.ucr.ie.dto.FacturaDTO;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;

```

```

import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service
public class FacturaService {

    private final FacturaRepository facturaRepository;

    public FacturaService(FacturaRepository repository) {
        this.facturaRepository = repository;
    }

    @Transactional(readOnly = true)
    public Page<FacturaDTO> listarPaginado(int page,
                                           int size,
                                           String sortBy,
                                           String dir,
                                           String cliente) {

        Sort.Direction direction =
            dir.equalsIgnoreCase("DESC") ?
            Sort.Direction.DESC : Sort.Direction.ASC;

        Pageable pageable =
            PageRequest.of(page, size,
                           Sort.by(direction, sortBy));

        Page<Factura> facturasPage;
        if (cliente != null && !cliente.trim().isEmpty()) {
            facturasPage = facturaRepository
                .findByClienteContainingIgnoreCase(
                    cliente, pageable
                );
        } else {
            facturasPage =
                facturaRepository.findAll(pageable);
        }

        return facturasPage.map(f -> new FacturaDTO(
            f.getId(), f.getNumeroFactura(),
            f.getCliente(), f.getMontoTotal(),
            f.getFecha(), f.getEstado()
        ));
    }
}

```

```

    @Transactional(readOnly = true)
    public List<FacturaDTO> listarViaStoredProcedure(String estado) {
        List<Factura> lista =
            facturaRepository.obtenerPorEstado(estado);

        return lista.stream()
            .map(f -> new FacturaDTO(
                f.getId(), f.getNumeroFactura(),
                f.getCliente(), f.getMontoTotal(),
                f.getFecha(), f.getEstado()
            ))
            .toList();
    }
}

```

Paso 4: Controlador RESTful (FacturaController.java)

Cree el controlador FacturaController.java en controller:

```

package cr.ac.ucr.ie.controller;

import cr.ac.ucr.ie.business.FacturaService;
import cr.ac.ucr.ie.dto.FacturaDTO;
import org.springframework.data.domain.Page;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/v1/facturas")
@CrossOrigin(origins = "*")
public class FacturaController {

    private final FacturaService facturaService;

    public FacturaController(FacturaService facturaService) {
        this.facturaService = facturaService;
    }

    @GetMapping

```

```

    public ResponseEntity<Page<FacturaDTO>> listar(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "5") int size,
        @RequestParam(defaultValue = "fecha") String sortBy,
        @RequestParam(defaultValue = "DESC") String direction,
        @RequestParam(required = false) String cliente) {
        Page<FacturaDTO> resultado =
            facturaService.listarPaginado(
                page, size, sortBy, direction, cliente
            );
        return ResponseEntity.ok(resultado);
    }

    @GetMapping("/procedimiento/{estado}")
    public ResponseEntity<List<FacturaDTO>> listarSP(
        @PathVariable String estado) {
        List<FacturaDTO> resultado =
            facturaService.listarViaStoredProcedure(estado);
        return ResponseEntity.ok(resultado);
    }
}

```

Paso 5: Interfaz de Usuario Cliente (HTML5 + CSS3 + Vanilla JavaScript)

Cree el archivo `src/main/resources/static/index.html`:

```

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
        content="width=device-width, initial-scale=1.0">
    <title>Gestion de Facturas - Paginacion</title>
    <style>
        :root {
            --bg-color: #0f172a;
            --card-bg: #1e293b;
            --accent-color: #38bdf8;
            --text-color: #f8fafc;
            --border-color: #334155;

```

```

    }
}

body {
    font-family: 'Segoe UI', sans-serif;
    background-color: var(--bg-color);
    color: var(--text-color);
    margin: 0;
    padding: 20px;
}

.container {
    max-width: 900px;
    margin: 0 auto;
    background-color: var(--card-bg);
    padding: 20px;
    border-radius: 10px;
}

h1 {
    color: var(--accent-color);
    font-size: 20px;
    border-bottom: 2px solid var(--border-color);
    padding-bottom: 8px;
}

.filter-bar {
    display: flex;
    gap: 10px;
    margin-bottom: 15px;
    flex-wrap: wrap;
}

input, select, button {
    padding: 8px 12px;
    border-radius: 5px;
    border: 1px solid var(--border-color);
    background-color: #0f172a;
    color: var(--text-color);
    font-size: 13px;
}

button {
    background-color: #0284c7;
    color: white;
    cursor: pointer;
    border: none;
}

```

```

    }
    button:disabled {
        opacity: 0.4;
        cursor: not-allowed;
    }

    table {
        width: 100%;
        border-collapse: collapse;
        margin-bottom: 15px;
    }

    th, td {
        padding: 10px;
        text-align: left;
        border-bottom: 1px solid var(--border-color);
        font-size: 13px;
    }

    th {
        background-color: #0f172a;
        color: var(--accent-color);
    }

    .pagination-controls {
        display: flex;
        justify-content: space-between;
        align-items: center;
        font-size: 13px;
    }
</style>
</head>
<body>

<div class="container">
    <h1>Facturacion Empresarial</h1>

    <div class="filter-bar">
        <input type="text" id="inputCliente"
            placeholder="Buscar cliente...">
        <button onclick="buscarCliente()">Buscar</button>

        <select id="selectSP"
            onchange="ejecutarSP()">
            <option value="">-- Probar SP --</option>

```

```

        <option value="PAGADA">SP: PAGADA</option>
        <option value="PENDIENTE">SP: PENDIENTE</option>
    </select>
    <br>
    <select id="selectSize"
        onchange="cambiarTamano()">
        <option value="5" selected>5 por pag</option>
        <option value="10">10 por pag</option>
    </select>
</div>
<br>
<table>
    <thead>
        <tr>
            <th>No. Factura</th>
            <th>Cliente</th>
            <th>Monto</th>
            <th>Fecha</th>
            <th>Estado</th>
        </tr>
    </thead>
    <tbody id="tablaBody"></tbody>
</table>
<br>
<div class="pagination-controls">
    <div id="infoPagina">Cargando...</div>
    <div>
        <button id="btnAnterior"
            onclick="paginaAnterior()">&lt; Ant</button>
        <button id="btnSiguiente"
            onclick="paginaSiguiente()">Sig &gt;</button>
    </div>
</div>
</div>
<br>
<script>
    let currentPage = 0;
    let pageSize = 5;
    let totalPages = 0;
    let filtroCliente = "";

    document.addEventListener("DOMContentLoaded", () => {
        cargar();
    });

    async function cargar() {

```

```

    let url = `/api/v1/facturas?page=${currentPage}` +
      `&size=${pageSize}`;
    if (filtroCliente) {
      url += `&cliente=${encodeURIComponent(filtroCliente)}`;
    }
    const resp = await fetch(url);
    const data = await resp.json();
    renderTabla(data.content);
    renderPaginador(data);
  }

  function renderTabla(lista) {
    const body = document.getElementById("tablaBody");
    body.innerHTML = "";
    lista.forEach(f => {
      const tr = document.createElement("tr");
      tr.innerHTML = `
        <td><strong>${f.numeroFactura}</strong></td>
        <td>${f.cliente}</td>
        <td>${f.montoTotal.toFixed(2)}</td>
        <td>${f.fecha}</td>
        <td>${f.estado}</td>
      `;
      body.appendChild(tr);
    });
  }

  function renderPaginador(data) {
    totalPages = data.totalPages;
    const info = `Pagina ${data.number + 1} ` +
      `de ${data.totalPages} ` +
      `(Total: ${data.totalElements})`;
    document.getElementById("infoPagina").innerText = info;
    document.getElementById("btnAnterior").disabled =
      data.first;
    document.getElementById("btnSiguiente").disabled =
      data.last;
  }

  function paginaAnterior() {
    if (currentPage > 0) {
      currentPage--;
      cargar();
    }
  }
}

```

```

function paginaSiguiente() {
    if (currentPage < totalPages - 1) {
        currentPage++;
        cargar();
    }
}

function cambiarTamano() {
    pageSize =
        parseInt(document.getElementById("selectSize").value);
    currentPage = 0;
    cargar();
}

function buscarCliente() {
    filtroCliente =
        document.getElementById("inputCliente").value;
    currentPage = 0;
    cargar();
}

async function ejecutarSP() {
    const est = document.getElementById("selectSP").value;
    if (!est) { cargar(); return; }
    const resp =
        await fetch(`/api/v1/facturas/procedimiento/${est}`);
    const data = await resp.json();
    renderTabla(data);
    document.getElementById("infoPagina").innerText =
        `Resultados SP: ${data.length} registros`;
    document.getElementById("btnAnterior").disabled = true;
    document.getElementById("btnSiguiente").disabled = true;
}
</script>
</body>
</html>

```

Parte 2: Seccion de Depuracion de Errores Comunes

Error 1: Desajuste de Índice Base 0 en Spring Data (`IndexOutOfBoundsException`)

Guía de Depuración:

- [] Verifique la URL emitida en la pestaña **Network** (F12) del navegador.
 - [] Observe que Spring Data interpreta `page = 0` como la primera página.
 - [] Inspeccione el controlador Java y asegúrese de que `@RequestParam default = "0"` coincida con el script.
 - [] En JavaScript, muestre al usuario `pageData.number + 1` mientras envía `pageData.number` a la API.
-

Error 2: Advertencia de Paginación en Memoria (HHH000104)

Guía de Depuración:

- [] Revise sus consultas JPQL en `FacturaRepository.java`.
 - [] Confirme que no está utilizando `JOIN FETCH` sobre colecciones `@OneToMany` con `Pageable`.
 - [] Para solucionar este error, aplique `@EntityGraph(attributePaths = {"detalles"})` sobre el repositorio.
-

Parte 3: Reto Autonomo

1. **Nuevo Stored Procedure:** Cree `SP_CALCULAR_TOTALES_FACTURACION` con parámetro `IN pEstado` y parámetros `OUT pTotal` y `pMonto`.
2. **Mapeo y Endpoint:** Exponga el método en `FacturaService` y el controlador `GET /api/v1/facturas/resumen/{estado}`.
3. **UI:** Agregue tarjetas dinámicas en HTML que muestren los totales acumulados devueltos por el procedimiento almacenado.